

FreeRTOS Command Shell

Includes Bare-Metal STM32F411RE Projects with UART, I2C, PWM, SPI, and FreeRTOS integration

6/30/2026

Nathaniel Ross

Table of Contents

Project 5 - FreeRTOS Integration	3
Project 4 - SPI RFID Reader	6
Project 3 - PWM/Timer Controller	16
Project 2 - Bare-Metal I2C Sensor Dashboard	26
Project 1 - UART Shell	40

Project 5 - FreeRTOS Integration

Nathaniel Ross

Overview:

This project integrated all of the peripherals developed in the previous four projects—including the UART command shell, I2C-based MPU 6500 sensor dashboard, PWM LED timer control, and SPI-based MFRC522 RFID reader—into a single system running on FreeRTOS on my STM32 F411RE NUCLEO. The goal was to convert the project's sequential main loop into four concurrent, preemptively scheduled tasks, with each task being responsible for one subsystem.

Task A: FreeRTOS Setup

FreeRTOS was enabled through STM32CubeMX using the CMSIS_V2 interface. Enabling FreeRTOS required moving the HAL timer off SysTick (which FreeRTOS uses for its own scheduler) and onto TIM1. The four tasks mirrored the previous four projects:

1. uart_task (high priority)
2. mfrc522_task (above normal priority)
3. mpu_task (normal priority)
4. pulse_task (low priority)

Task priorities were assigned based on timing sensitivity. For example, the UART task had the highest priority because delayed readings could result in loss of user inputs and incorrect sensor reading displays. On the other hand, the LED pulse task was given the lowest priority because an occasional delayed execution has no lasting consequence on the user experience.

Task B: Synchronize RTOS

There was one shared resource that required protection: the UART peripheral (used by all four tasks for output). A **recursive mutex** (uart_mutex) was created to protect all UART transmissions. The mutex was recursive because uart_task would acquire the mutex and dispatch a command, where the read and stream commands called print_mpu_data(), which also needed to acquire the mutex. When I initially implemented uart_mutex nonrecursively, the program would **deadlock**, meaning uart_task was blocking print_mpu_data() from executing.

Binary **semaphores** were used to replace the polling flags data_ready and pulse_ready that were used previously. Timers TIM2 and TIM4 ISRs were updated to call xSemaphoreGiveFromISR() and portYIELD_FROM_ISR(). This meant mpu_task and pulse_task now block with zero CPU usage until their timer fires instead of loop checking their flags.

Task C: Debugging

Introducing FreeRTOS to the system required fixing many bugs iteratively as more tasks were integrated. To handle this effectively, I would introduce one task to the system, debug until it worked as expected, and then introduce the next. Significant problems I ran into include:

Interrupt-driven UART RX stopped working: After enabling FreeRTOS, the UART receive interrupt would not fire, and therefore I could not type into the command shell. To fix this, I went through each part of the UART TX and RX process that was currently in my code. I found that waiting to arm the receive interrupt until after `uart_task` was running would solve the issue. The problem was that arming the interrupt before the scheduler started would lockup the system.

System halts from floating points with `snprintf()`: When attempting to print the MPU sensor data, the system would completely halt. This was found when testing the read command. To debug this issue, I placed debug print statements around the read command and `print_mpu_data()` function, and found that the system halted right before sensor data was output. I knew that the I2C communication was not an issue after noting that the raw data printed from the MPU (integers) were correct. After doing some research, I determined that the issue came from combining floats with `snprintf()`. While this did not affect my previous project because I manually enabled floats with `snprintf()`, the reason it didn't work this time was different. Combining floats and `snprintf()` consumes much more stack space than I anticipated, resulting in a stack overflow. Instead of increasing the stack sizes indefinitely, I used fixed-point integer arithmetic. This meant multiplying raw data by 100 and then printing the raw data divided by 100 and taking the remainder for the first two decimal digits. This used data type `uint32_t` instead of `float`, and the read command worked perfectly once implemented.

Timer interrupt priorities conflicted with FreeRTOS semaphore calls: One of the most frequent issues I encountered when introducing FreeRTOS was with priority conflicts. When enabling the `mpu_task`, I found that streaming the sensor data would not work. I immediately knew it was not an I2C communication issue because the sensor read command worked, meaning the issue had to do with either the timer or semaphore used. The issue ended up being the timer's default priority, zero, was higher (numerically lower) than the FreeRTOS max syscall interrupt priority of five. To fix this issue, and the same issue that occurred with the `pulse_task`, TIM2 and TIM4 had their interrupt priorities set to six.

Many sequential commands would freeze the system: When I would have many tasks running together and run commands very quickly, the system would occasionally lock up. To diagnose this issue, I implemented a stack overflow hook. It determined that there was a stack overflow, and after increasing a few of the tasks' stack sizes, the system started working perfectly. In previous projects, every function shared a large stack, so deep call chains worked fine as the stack size was effectively unlimited. Because FreeRTOS allocates each task a small, fixed-size stack, the same call chains had to be accounted for. Stack sizes were increased based on their respective needs, including their deepest call chain.

Results:

The completed system runs all four tasks concurrently and indefinitely with no lockups. The UART shell remains responsive, MPU data streams, LED pulses or dims, and the MFRC522 polls for RFID cards. Additionally, further refactoring of the project was done to split task-specific logic into `uart.c`, `mpu.c`, `mfr522.c`, and `tim_int.c` files. This made the `main.c` file mostly initializations and little logic.

What was learned:

- Preemptive scheduling with FreeRTOS fundamentally changes how hardware resources are treated. While code can behave correctly in a single-threaded bare-metal loop, it can deadlock, corrupt output, or result in a stack overflow under a RTOS.
- Mutexes are used to signify ownership of a resource, with this project's example being UART communication. Semaphores are used to coordinate task execution and signal when a thread can use a resource.
- Floating point formatting with `snprintf` is very expensive and introduces a commonly occurring issue of stack overflow in a RTOS
- Each task owns its own private, fixed-sized stack. This stack must be large enough to accommodate deep call chains and HAL functions used by the task.
- Overall, this project was incredibly helpful in introducing and reinforcing embedded systems topics. Something I am very proud of is that this project utilizes UART, I2C, and SPI communication protocols, with I2C and SPI being initialized with bare-metal programming. This has enhanced my ability to configure registers, poll flags, and follow/read datasheets to find necessary information. There are many more things I learned, which are discussed in greater detail in the rest of the document, that includes polling, interrupts, setting up a ring buffer, using HAL, writing bare-metal code, properties and initialization of I2C, open-drain vs. push-pull, pull-down vs. pull-up resistors, GPIO registers, timers, blocking vs. non-blocking, PWM, PWM frequency calculation, properties and initialization of SPI, full-duplex vs. half-duplex, atomic processes, and debouncing.

The rest of this document follows the previous four projects and includes what was done, how it was done, and what I learned from each. Each project follows a format of what I had to do, relevant code, results, and knowledge gained for significant parts of the project.

Project 4 - SPI RFID Reader

Nathaniel Ross

Task A: SPI Init

Do: Create C and Header files to initialize the SPI protocol.

Code:

spl_init.c:

```
void spl_init(void)
{
    // configure clocks
    RCC->AHB1ENR |= (1 << 1);           // GPIOB clock enable
    RCC->AHB1ENR |= (1 << 2);           // GPIOC clock enable
    RCC->APB2ENR |= (1 << 12);          // SPI1 clock enable
    // configure PB3, PB4, PB5
    GPIOB->MODER &= ~(0b111111 << 6);  // clear PB3,PB4,PB5
    GPIOB->MODER |= (0b101010 << 6);    // AF mode for PB3,PB4,PB5
    GPIOB->AFR[0] &= ~(0xFFF << 12);    // clear AF selections
    GPIOB->AFR[0] |= (0x555 << 12);      // set all AF05
    GPIOB->OTYPER &= ~(0b111 << 3);     // push-pull output type
    GPIOB->PUPDR &= ~(0b111111 << 6);  // no pull-up, pull-down
    GPIOB->OSPEEDR |= (0b111111 << 6);  // high speed
    // configure PB6, PC7
    GPIOB->MODER &= ~(0b11 << 12);      // clear PB6
    GPIOC->MODER &= ~(0b11 << 14);      // clear PC7
    GPIOB->MODER |= (0b01 << 12);       // output mode for PB6
    GPIOC->MODER |= (0b01 << 14);       // output mode for PC7
    GPIOB->OTYPER &= ~(1 << 6);         // push-pull output type
    GPIOC->OTYPER &= ~(1 << 7);         // push-pull output type
    GPIOB->PUPDR &= ~(0b11 << 12);      // no pull-up, pull-down
    GPIOC->PUPDR &= ~(0b11 << 14);      // no pull-up, pull-down
    GPIOB->BSRR = (1 << 6);             // idle CS high
    GPIOC->BSRR = (1 << 7);             // idle RST high
    // configure master mode
    SPI1->CR1 = 0;                      // clear configuration
    SPI1->CR1 |= (1 << 2);              // set master configuration
    SPI1->CR1 &= ~(0b111 << 3);
    SPI1->CR1 |= (0b011 << 3);          // baud rate = 84MHz/16 = 5.25 MHz
    SPI1->CR1 &= ~(1 << 1);             // CPOL = 0, clock idle low
    SPI1->CR1 &= ~(1 << 0);             // CPHA = 0, sample on first edge
    SPI1->CR1 &= ~(1 << 7);             // MSB transmitted first
    SPI1->CR1 &= ~(1 << 11);           // 8-bit data frame format
    SPI1->CR1 |= (1 << 9);              // software slave management
    SPI1->CR1 |= (1 << 8);              // internal NSS high (SSI)
    SPI1->CR1 |= (1 << 6);             // enable SPI
}

uint8_t spl_transfer(uint8_t tx_data, uint8_t *rx_data)
```

```

{
    uint32_t timeout = 10000;
    while(!(SPI1->SR & (1 << 1)))          // wait for TXE to be set
    {
        if(timeout-- == 0)
            return SPI_ERROR;
    }
    *((volatile uint8_t*)&SPI1->DR) = tx_data; // write byte to data register
    timeout = 10000;
    while(!(SPI1->SR & (1 << 0)))          // wait for RXNE to be set
    {
        if(timeout-- == 0)
            return SPI_ERROR;
    }
    *rx_data = *((volatile uint8_t*)&SPI1->DR); // read received byte
    return SPI_OK;
}
uint8_t spil_receive(uint8_t *rx_data)
{
    return spil_transfer(0xFF, rx_data); // dummy byte to receive real byte
}

```

Results:

20:21:39 Build Finished. 0 errors, 0 warnings. (took 1s.7ms)

None so far, compiles correctly

Learned:

- How to initialize the SPI protocol using bare-metal programming in Embedded C
- Configuring the GPIO pins was very similar to configuring the I2C protocol: Enable the GPIOB/C clock, enable SPI1 clock, pins for alternate function mode, set the alternate functions based on the STM32F411xE datasheet, set push-pull
- The SPI protocol is different from I2C because it is **full-duplex** (master and slave send data to each other simultaneously) while I2C is **half-duplex** (master and slave take turns sending data).
- SPI pins use **push-pull** instead of **open-drain**. Push-pull is much faster than open-drain because it uses two transistors to drive HIGH and LOW, while open-drain uses one transistor to drive LOW and requires pull-up resistors to drive HIGH. SPI uses push-pull because it is used for high-speed data transfer (in our case, RF signals) and only connects to one device. To contrast, I2C uses open-drain because it can connect to many devices on the same line.
- The CS and RST pins for the SPI are regular GPIO pins, however they are updated using the BSRR (bit set reset register) because BSRR is **atomic** (basically instant). This is crucial in SPI communication, because data is transferring very fast, and if the chip select is not updated in time, a data transaction can be completely lost.
- The baud rate was set to 5.25 MHz (84Mhz/16) because the MFRC522 datasheet says its maximum communication rate with SPI is 10MHz
- CPOL (clock polarity) and CPHA (clock phase) were both set to zero. This means that the clock is LOW when idle, and the first data sample is on the first clock edge
- When transferring and receiving data, we have to wait for the transmit buffer to be empty and the receive buffer to be not empty. So even if there is no data to be sent, in order to receive data there has to be dummy values sent. This is part of the two-way communication between the Master and Slave.

Task B: Verify MFRC522 Device

Task: Initialize the MFRC522 device and read its Version Register to ensure the device is working.

Code:

mfrc522.c:

```
#include "mfrc522.h"
#include <stdio.h>
uint8_t mfrc522_init(void)
{
    GPIOC->BSRR = (1 << 23);          // hold RST LOW
    HAL_Delay(1);
    GPIOC->BSRR = (1 << 7);            // set RST high
    HAL_Delay(50);                     // allow oscillator to stabilize
    uint8_t result;
    uint8_t status = mfrc522_read_reg(VERSION_REG, &result);
    if(status == SPI_ERROR)
        return SPI_ERROR;
    // make sure chip is working
    if(result != 0x00 && result != 0xFF)
        return SPI_OK;

    return SPI_ERROR;
}
uint8_t mfrc522_read_reg(uint8_t reg, uint8_t *result)
{
    cs_low();                          // pull CS low
    uint8_t rx_dummy;
    // transfer address byte
    // read: shift left, clear bit 0, set bit 7
    uint8_t status = spil_transfer( ((reg << 1) & 0x7E) | 0x80, &rx_dummy);
    if(status == SPI_ERROR)
    {
        cs_high();                    // pull CS high
        return SPI_ERROR;
    }
    // receive data byte
    status = spil_receive(result);
    if(status == SPI_ERROR)
    {
        cs_high();                    // pull CS high
        return SPI_ERROR;
    }
    cs_high();                          // pull CS high
    return SPI_OK;
}
uint8_t mfrc522_write_reg(uint8_t reg, uint8_t value)
{

```

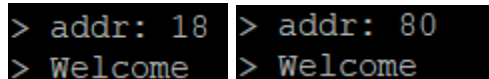
```

cs_low(); // pull CS low
uint8_t rx_dummy;
// transfer address byte
// write: shift left, clear bits 0 and 7
uint8_t status = spil_transfer((reg << 1) & 0x7E, &rx_dummy);
if(status == SPI_ERROR)
{
    cs_high();
    return SPI_ERROR;
}

// send data byte
status = spil_transfer(value, &rx_dummy);
if(status == SPI_ERROR)
{
    cs_high();
    return SPI_ERROR;
}
cs_high();
return SPI_OK;
}

```

Results:



```

> addr: 18 > addr: 80
> Welcome > Welcome

```

Printing the VersionReg gave 0x18. This is different from the expected 0x91 or 0x92 because the version I have is likely a fake (I got it from an ELEGOO Arduino Kit). Importantly, it doesn't show 0x00/0xFF, and other key registers (ex: TxControlReg) return correct reset values (0x80).

Learned:

- How to use the SPI transfer and receive functions I made previously to write and read to the MFRC522
- When transferring the address byte, reading and writing have different protocols according to the MFRC522's datasheet: To read, shift the address left 1 bit, clear bit 0, and set bit 7. To write, shift the address left 1 bit and clear bits 0 and 7.
- CS has to be pulled low at the start of each read/write, and pulled high at the end of each read/write
- When initializing the MFRC522, RST has to be held LOW for at least 100ns to complete reset, then RST has to be set high with about a 40ms period to allow the internal oscillator to stabilize and the device to be configured. This is reflected in the `mfr522_init()` function with a `HAL_Delay(1)` after RST LOW and a `HAL_Delay(50)` after RST HIGH.
- The MFRC522 I am using is likely a fake/copycat because its VersionReg (which is supposed to have reset value of 0x91/0x92) has a reset value of 0x18

Task C: Card UID Print

Do: Add a function to the main loop that prints the UID of a card when it is scanned by the MFRC522.

Code:

mfrc522.c:

```
uint8_t mfrc522_read_uid(uint8_t *uid)
{
    uint8_t atqa;

    if(mfrc522_detect_card(&atqa) == SPI_ERROR) // send REQA byte
        return SPI_ERROR;

    mfrc522_write_reg(COMMAND_REG, 0x00);           // idle
    mfrc522_write_reg(COM_IRQ_REG, 0x7F);          // clear IRQ flags
    mfrc522_write_reg(FIFO_LEVEL_REG, 0x80);        // flush FIFO
    mfrc522_write_reg(FIFO_DATA_REG, 0x93);         // anticollision command
    mfrc522_write_reg(FIFO_DATA_REG, 0x20);         // anticollision command
    mfrc522_write_reg(COMMAND_REG, 0x0C);           // transceive
    mfrc522_write_reg(BITFRAME_REG, 0x00);         // full 8-bit frame
    mfrc522_write_reg(BITFRAME_REG, 0x80);         // start transmission

    uint32_t timeout = 10000;
    uint8_t val = 0;
    while(!(val & (1 << 5)))                        // wait for RxIRQ set
    {
        if(timeout-- == 0)
            return SPI_ERROR;
        if(mfrc522_read_reg(COM_IRQ_REG, &val) == SPI_ERROR)
            return SPI_ERROR;
    }

    uint8_t bcc;                                    // FIFO read
    for(int i = 0; i < 5; i++)
    {
        if(i < 4)
            mfrc522_read_reg(FIFO_DATA_REG, &uid[i]);
        else
            mfrc522_read_reg(FIFO_DATA_REG, &bcc);
    }

    if((uid[0] ^ uid[1] ^ uid[2] ^ uid[3]) != bcc) // bit check
        return SPI_ERROR;
    return SPI_OK;
}
```

```

uint8_t mfrc522_detect_card(uint8_t *atqa)
{
    mfrc522_write_reg(COMMAND_REG, 0x00);           // idle
    mfrc522_write_reg(COM_IRQ_REG, 0x7F);          // clear IRQ flags
    mfrc522_write_reg(FIFO_LEVEL_REG, 0x80);        // flush FIFO
    mfrc522_write_reg(FIFO_DATA_REG, 0x26);         // load REQA byte
    mfrc522_write_reg(COMMAND_REG, 0x0C);          // transceive
    mfrc522_write_reg(BITFRAME_REG, 0x87);         // start transmission

    uint32_t timeout = 10000;
    uint8_t val = 0;
    while(!(val & (1 << 5)))                        // wait for RxIRQ set
    {
        if(timeout-- == 0)
            return SPI_ERROR;
        if(mfrc522_read_reg(COM_IRQ_REG, &val) == SPI_ERROR)
            return SPI_ERROR;
    }

    if(mfrc522_read_reg(FIFO_DATA_REG, atqa) == SPI_ERROR)
        return SPI_ERROR;

    return SPI_OK;
}

```

main.c:

```

void handle_mfrc522(void)
{
    uint8_t uid[4];
    char msg[16];
    if(mfrc_state == 0 && mfrc522_read_uid(uid) == SPI_OK)
    {
        snprintf(msg, sizeof(msg), "> uid: ");
        HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
        for(int i = 0; i < 4; i++)
        {
            snprintf(msg, sizeof(msg), "%X", uid[i]);
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
        }
        snprintf(msg, sizeof(msg), "\r\n");
        HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
        mfrc_state = 1;
        last_detected_tick = HAL_GetTick();
    }
    else if(mfrc_state == 1 && HAL_GetTick() - last_detected_tick > 2000)
        mfrc_state = 0;
}

```

Results:

```
> uid: E757166  
> uid: F1DB161
```

Blue card first, white card second

Learned:

- How to detect a card scan and read the UID of a card for the MFRC522. To do this, I had to read through the documentation of the MFRC522 and determine which device registers needed to be altered. The process of doing so was specific to the MFRC, and it involved clearing the command register, clearing interrupt requests, flushing the device's FIFO queue, loading the REQA (request guard) byte, starting the transmission, and then receiving data from the device.
- The process of transmitting and receiving from the MFRC522 followed the ISO 14443 / RFID protocol, with the use of REQA, ATQA, anticollision, and UID. In the "detect_card" function, the ATQA was found to verify a reading.
- In all, learning this protocol involved reading the datasheet, understanding registers, building a driver for this device, and verifying each step as I went
- Handling the card scanning in the main loop required **debouncing**. To explain, when a card would be tapped to the sensor, it would initially register dozens of scans per second. This is not the intended behavior, so debouncing was implemented in software. This debouncing followed a state machine of the card's state, with state 0 being a scan and 1 being no scan. A counter was implemented that, if there are at least 2000 clock ticks between scans, the scan is valid. This removed the console being flooded with scans and registers one scan every 2 seconds.

Task D: Card Authentication

Do: Add functionality that turns on a green LED when the correct card is scanned and turns on a red LED when the incorrect card is scanned.

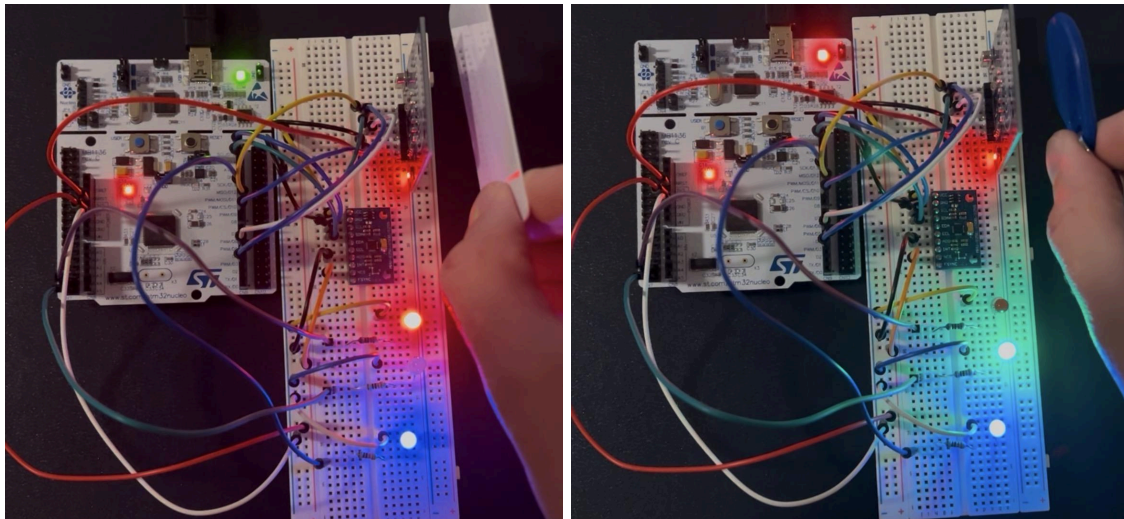
Code:

main.c:

```
void check_uid(uint8_t *uid)
{
    char msg[64];
    if(memcmp(uid, correct_uid, 4) == 0)
    {
        GPIOC->ODR &= ~(1 << 2);    // turn red led off
        GPIOC->ODR |= (1 << 3);      // turn green led on
        snprintf(msg, sizeof(msg), "> Authentication Successful\r\n");
        HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);

    }
    else
    {
        GPIOC->ODR &= ~(1 << 3);      // turn green led off
        GPIOC->ODR |= (1 << 2);      // turn red led on
        snprintf(msg, sizeof(msg), "> Authentication Denied\r\n");
        HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
    }
}
```

Results:



```
> Authentication Denied
> Authentication Successful
auth_leds_off
> Authorization LEDs off
```

I also added a command to turn off the LEDs because, once a card is scanned, at least one is on at all times.

Learned:

- This part of the project reinforced basic embedded systems concepts. This means configuring the GPIO pins for the red and green LEDs, wiring them correctly with current-limiting resistors, and using pointers correctly for passing variables.
- A small bug was fixed: when the user typed the backspace in the UART Shell, the backspace character would become part of the command, resulting in an “unknown command” error. This is because the software checked for “\b”, while the terminal sent “0x7F” (delete) instead. Now both cases are accounted for.

What was learned overall:

- Basics of the SPI communication was learned and implemented in this project. SPI is a **full-duplex** protocol (communicating both ways), different from I2C which is half-duplex. SPI pins use **push-pull** instead of open drain because push-pull is much faster. This is good for high-speed data transfer, such as the RF device used in this project.
- Basic embedded system concepts were reinforced, such as configuring clocks, configuring GPIO pins for intended behavior, reading datasheets to learn intended behavior, and setting or clearing bits in registers.
- For SPI communication, because it is both ways (master to slave and slave to master), there sometimes has to be dummy information sent one way in order to receive information from the other
- Setting up the MFRC522 was unique to the information found in its datasheet, however it did follow the ISO 14443 / RFID protocol with the use of REQA, ATQA, anticollision, and UID.
- **Debouncing** was used for the card scanning feature. Instead of a card being read dozens of times per scan, a counter and state machine was implemented to have only one read per card scan.
- Overall, this project reinforced embedded systems topics covered across all four projects so far. This includes configuring GPIO pins, bit shifting to configure registers, using pointers, and following the STM32 F411RE datasheet and datasheets of devices added to this project. Further improvements of this project could be better pin and breadboard management. Currently, the wiring is fairly jumbled and the breadboard is slightly disorganized. In the future, these pins and layout could be thought out before implementation began. There is still potential to add more devices or protocols to this project, however further work will focus on using FreeRTOS with the current implementation.

Project 3 - PWM/Timer Controller

Nathaniel Ross

Task A: PWM Timer Init

Do: Initialize the PWM timer and show it works by wiring an LED to the STM32

Code:

```
tim3_pwm_init.c:
#include "tim3_pwm_init.h"

void tm3_init(void)
{
    // 1. peripheral clock enables
    RCC->AHB1ENR |= 1;           // GPIOA
    RCC->APB1ENR |= (1 << 1);    // TIM3

    // 2. configure GPIOA as alternate function
    GPIOA->MODER &= ~(0b11 << 12);
    GPIOA->MODER |= (0b10 << 12); // AF mode

    // set to push-pull
    GPIOA->OTYPER &= ~(1 << 6);

    // set to no pull-up pull-down
    GPIOA->PUPDR &= ~(0b11 << 12);

    // configure AF02 for PA6
    GPIOA->AFR[0] &= ~(0xF << 24);
    GPIOA->AFR[0] |= (0x2 << 24);

    // 3. configure timer base
    TIM3->PSC &= ~(0xFFFF);
    TIM3->PSC |= (0b101000111); // set PSC to 327
    TIM3->ARR &= ~(0xFFFF);
    TIM3->ARR |= (0xFF);        // set ARR to 255

    // 4. configure PWM channel
    TIM3->CCMR1 &= ~(0b111 << 4);
    TIM3->CCMR1 |= (0b110 << 4); // PWM Mode 1
    TIM3->CCMR1 |= (1 << 3);      // enable preload register
    TIM3->CCR1 &= ~(0xFFFF);
    TIM3->CCR1 |= (0x80);         // CCR1 = 128, ~50% duty cycle

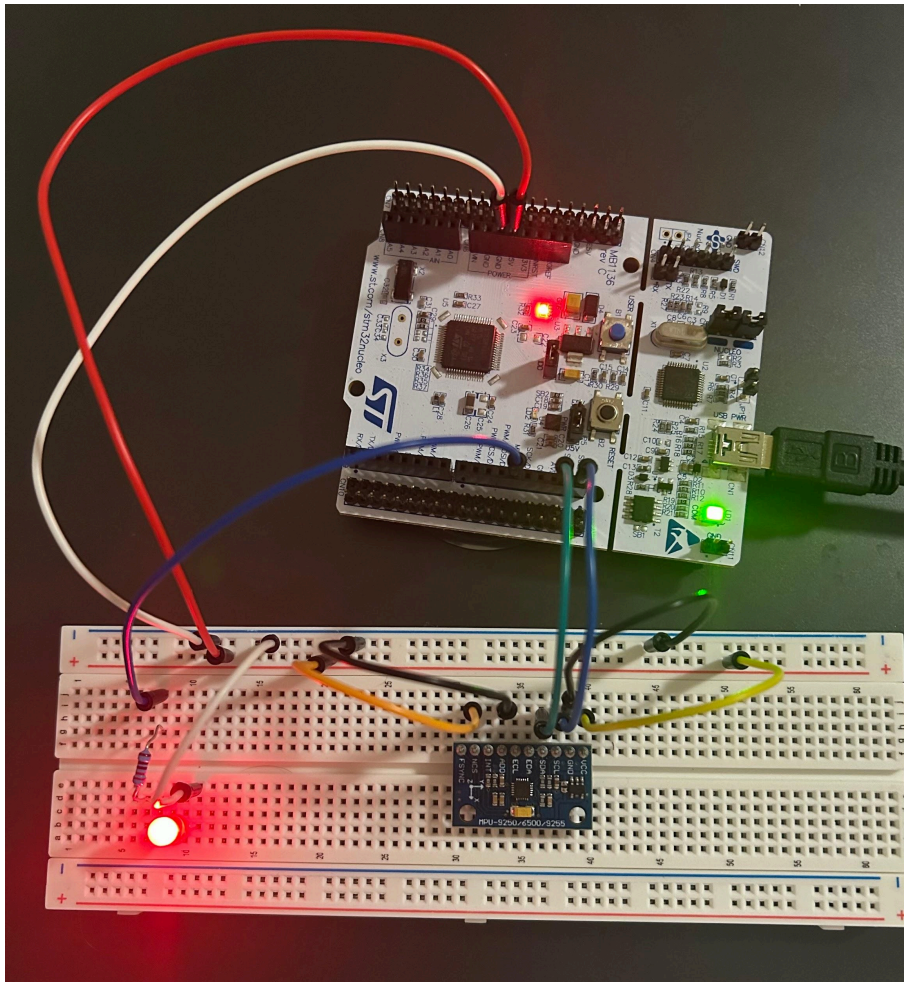
    // 5. enable output channel
    TIM3->CCER &= ~(0b11);
    TIM3->CCER |= (0b01);         // enable channel active high
```

```
// 6. enable auto-reload preload
TIM3->CR1 |= (1 << 7);

// 7. force update for PSC/ARR/preload values
TIM3->EGR |= 0b1; // set UG (update generation)

// 8. start timer
TIM3->CR1 |= 0b1; // set CEN (counter enable)
}
```

Results:



LED (bottom left) is lit up to about 50% brightness, as intended

Learned:

- How to generate a PWM signal on the STM32F411RE using the TIM3 peripheral and bare-metal programming
- Choose to use PA6 (D12 on board) for output to LED. This meant TIM3-CH1 was the timer and AF02 was the alternate function used based on the alternate function mapping table in the STM32F411RE datasheet
- A timer counts from 0 to ARR and compares the counter value with CCR1, generating the PWM
- PWM frequency determined by timer clock, prescaler (PSC), and auto-reload register (ARR / period): $f_{\text{PWM}} = f_{\text{CLK}} / ((\text{PSC}+1)(\text{ARR}+1))$
- There is a tradeoff between frequency accuracy and duty-cycle resolution: My PWM frequency was not able to hit exactly 1kHz, however I accepted that slight inaccuracy instead of raising the resolution
- Duty cycle is controlled by CCR1. In PWM Mode 1, raising CCR1 means higher duty cycle, while PWM Mode 2 is opposite
- Placing CCR1 into shadow register ensures duty cycle changes only occurs when PWM finishes period. ARPE enabled ensures ARR does not change before period.
- Setting the UG bit ensured that PSC, ARR, CCR1 were updated during initialization
- In general, I gained more experience reading STM32 reference manuals and datasheets to determine which bits, flags, and registers to manipulate. In turn, this has given me more understanding in embedded functionality by learning the purpose of each register involved during initialization

Task B: UART Command

Do: Add a command to the UART shell to change the LED's brightness by manipulating its duty cycle.

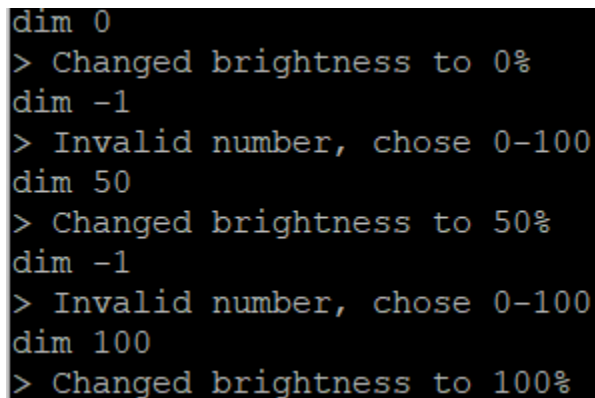
Code:

main.c:

```
int brightness;
// change attached LED brightness
else if(sscanf(cmd, "dim %d", &brightness) == 1)
{
    if(brightness >= 0 && brightness <= 100)
    {
        uint8_t val = brightness * 255 / 100;
        TIM3->CCR1 = val;

        char msg[64];
        snprintf(msg, sizeof(msg), "> Changed brightness to %d%\r\n",
brightness);
        HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
    }
    else
    {
        char msg[64];
        snprintf(msg, sizeof(msg), "> Invalid number, chose 0-100\r\n");
        HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
    }
}
```

Results:



```
dim 0
> Changed brightness to 0%
dim -1
> Invalid number, chose 0-100
dim 50
> Changed brightness to 50%
dim -1
> Invalid number, chose 0-100
dim 100
> Changed brightness to 100%
```

Learned:

- How to easily change duty cycle to change LED brightness with TIM3->CCR1
- Scale brightness level to correct value based on 0-255 range
- Parse an input command with sscanf

Task C: Stream data with interrupts

Do: Fix the “stream” command from the previous project to be non-blocking by using interrupts.

Code:

tim_init.c:

```
volatile uint8_t data_ready;
void tim2_init(void)
{
    RCC->APB1ENR |= (1 << 0);           // TIM2
    TIM2->CR1 = 0;                       // reset timer configuration
    TIM2->PSC = 1281;                   // PSC = 1281
    TIM2->ARR = 131044;                 // 65522 = 1 second
    TIM2->EGR |= 1;                     // set update generation
    TIM2->SR &= ~0b1;                   // clear UIF
    TIM2->DIER |= 0b1;                  // update interrupt enable
    TIM2->CR1 |= (1 << 7) | 1;          // ARPE, CEN
    NVIC->ISER[0] |= (1 << 28);         // interrupt controller enable
}
void TIM2_IRQHandler(void)
{
    TIM2->SR &= ~0b1;                   // clear UIF
    data_ready = 1;                     // signal timer period finished
}
```

main.c:

```
// non-blocking for streaming MPU sensor data
if(streaming && data_ready)
{
    print_mpu_data();
    data_ready = 0;
}
```

Results:

```
stream
> Streaming MPU Data
> Accel X: 0.10
> Accel Y: 0.09
> Accel Z: 1.04
> Temp (C): 22.96
> Gyro X: -3.66
> Gyro Y: 9.21
> Gyro Z: -1.49
> Accel X: 0.11
> Accel Y: 0.09
> Accel Z: 1.03
> Temp (C): 22.87
> Gyro X: -3.60
> Gyro Y: 9.51
> Gyro Z: -1.49
stop
> Stopping Data Stream
```

Learned:

- How to use an interrupt timer to read sensor data without polling
- How to manipulate the prescaler (PSC) and auto-reload (ARR) to have the sensor data read once every n seconds (set the period to n seconds)
- The “extern” key allows for variables to be used between files. When using the “data_ready” state variable, it was declared with the extern key in main.c to recognize it is also used in tim_init.c
- The update interrupt flag (UIF) should be cleared at the start of the interrupt handler instead of the end because it prevents new interrupts from being erased
- The timer has to be configured for interrupts with TIMx->DIER and NVIC has to be configured to allow an interrupt from that timer with NVIC->ISER. The specific bit in NVIC->ISER to be enabled for TIM2 is found in the stm32f411xe.h file
- Similar to the callback function for the UART interrupt communication, a timer uses an IRQHandler function to specify what to do once the timer period has elapsed

Task D: LED Pulse

Do: Add a pulse command that will use timer interrupts to smoothly pulse the LED until told to stop

Code:

main.c:

```
main() while loop
if(pulse_state != 0 && pulse_ready)
{
    // pulse based on state
    if(pulse_state == 1)
        pwm_val++;
    else if(pulse_state == 2)
        pwm_val--;

    // change state
    if(pwm_val == 0)
        pulse_state = 1;
    else if(pwm_val == 128)    // only lights up to 50% brightness
        pulse_state = 2;

    TIM3->CCR1 = pwm_val;
    pulse_ready = 0;
}

process_command() function
else if(strcmp((char*)cmd, "pulse") == 0)
{
    pwm_val = 0;
    TIM3->CCR1 = 0;
    pulse_state = 1;
    char msg[64];
    snprintf(msg, sizeof(msg), "> Pulsing LED\r\n");
    HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
}
else if (strcmp((char*)cmd, "stop_pulse") == 0)
{
    pulse_state = 0;
    char msg[64];
    snprintf(msg, sizeof(msg), "> Stopping LED Pulse\r\n");
    HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
}
```

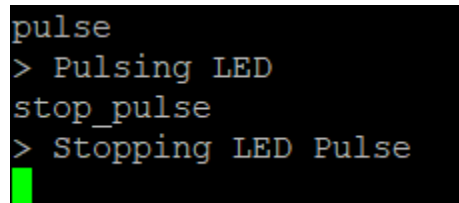
tim_init.c:

```

void tim4_init(void)
{
    RCC->APB1ENR |= (1 << 2);           // TIM 4 peripheral clock
    TIM4->CR1 = 0;
    TIM4->PSC = 1281;
    TIM4->ARR = (PULSE_INTERVAL_SEC * TICKS_PER_SECOND);
    TIM4->EGR |= 1;
    TIM4->SR &= ~0b1;
    TIM4->DIER |= 0b1;
    TIM4->CR1 |= (1 << 7) | 1;
    NVIC->ISER[0] |= (1 << 30);
}

void TIM4_IRQHandler(void)
{
    TIM4->SR &= ~0b1;
    pulse_ready = 1;
}

```

Results:


```

pulse
> Pulsing LED
stop_pulse
> Stopping LED Pulse

```

LED starts at 0, slowly lights up, and then slowly dims back to 0 repeatedly until stop_pulse ran

Learned:

- How to use a timer to pulse an LED. This command can also run concurrently with the “stream” command, meaning the LED will pulse and the MPU-6500 sensor data will be read to the console at the same time.
- How to create a state-machine for LED state: 0 = Off, 1 = Brighten, 2 = Dim
- Reinforced concepts learned in the previous task (streaming data with timer interrupts), including the use of the “extern” key, clearing the UIF flag at the start of the interrupt handler, configuring the timer for interrupts, configuring NVIC for interrupts from that timer, and writing an IRQHandler

Task E: Refactor Code

Do: Refactor the project code to be organized and easy to follow. It is currently a jumbled mess.

Results:

Turned the process_command() function into its own process_command.c file with separate functions for each command:

```

24 void process_command(uint8_t* cmd)
25 {
26     uint8_t newline[] = "\r\n";
27     HAL_UART_Transmit(&huart2, newline, 2, HAL_MAX_DELAY);
28     int brightness;
29
30     if(strcmp((char*)cmd, "led on") == 0)           // turns led on
31         cmd_led_on();
32
33     else if(strcmp((char*)cmd, "led off") == 0)      // turns led off
34         cmd_led_off();
35
36     else if(strcmp((char*)cmd, "status") == 0)       // fetches led status
37         cmd_status();
38
39     else if(strcmp((char*)cmd, "whoami") == 0)       // returns MPU's identity
40         cmd_whoami();
41
42     else if(strcmp((char*)cmd, "read") == 0)         // reads accel, temp, and gyro data from MPU
43         print_mpu_data();
44
45     else if(strcmp((char*)cmd, "stream") == 0)       // constantly streams MPU sensor data
46         cmd_stream();
47
48     else if(strcmp((char*)cmd, "stop") == 0)         // stops stream of MPU sensor data
49         cmd_stop();
50
51     else if(sscanf(cmd, "dim %d", &brightness) == 1) // change attached LED brightness
52         cmd_dim(brightness);
53
54     else if(strcmp((char*)cmd, "pulse") == 0)        // sets LED state to pulse
55         cmd_pulse();
56
57     else if (strcmp((char*)cmd, "stop_pulse") == 0)  // sets LED state to normal
58         cmd_stop_pulse();
59
60     else cmd_unknown();                             // unknown command entered
61 }

```

Completely cleaned up main.c while() loop:

```

/* USER CODE BEGIN WHILE */
while (1)
{
    handle_uart_cmd(cmd_buff, &cmd_index); // for user inputs
    stream_mpu();    // streams MPU sensor data
    pulse_led();    // pulsing command

    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}

```

Learned:

- How to refactor code to make it easier to use, follow, and show intended functionality
- Use the extern keyword to allow functions to be used between files

What was learned overall:

- How to generate a PWM signal using timers and bare-metal programming
- Alternate function GPIO mode has to be configured for PWM timers
- Timers work by counting from 0 to ARR, and comparing with CCR1. CCR1 controls the duty cycle by saying how much time the signal is high before going low
- PWM frequency follows the equation:
 - $f_PWM = f_CLK / ((PSC+1)(ARR+1))$
 - Inverting this can be used to find the period
- Timers can also be used for non-PWM functions, such as displaying sensor data. With this, PSC and ARR can be manipulated to have the sensor data read once every n seconds.
- A timer has to be configured for interrupts with DIER register, and the NVIC (interrupt controller) has to be configured for interrupts from that timer with the ISER register
- A timer uses an IRQHandler function to specify what to do once a timer period has elapsed, similar to the UART callback function
- The “extern” keyword allows for functions and variables to be used between different files
- Interrupts allow for multiple commands to be run concurrently: LED continuously pulses while the MPU-6500 continuously reads data
- How to refactor code to be easier to read, follow, and show intended functionality
- This project can still be improved in many ways. More sensors, devices, and commands could be added, but importantly more communication protocols could be used. In the future, a device that uses the SPI protocol could be added so that the project utilizes UART, I2C, and SPI protocols.

Project 2 - Bare-Metal I2C Sensor Dashboard

Nathaniel Ross

Task A: I2C Init

Do: Configure the I2C1 peripheral registers so the bus is ready to use.

Code:

main.c extras:

```
#include "i2c.h"
i2c1_init();
```

i2c.h

```
#ifndef I2C_H
#define I2C_H
```

```
#include "stm32f4xx.h"
```

```
void i2c1_init(void);
```

```
#endif
```

i2c.c

```
#include "i2c.h"
```

```
void i2c1_init(void) {
    // clock enables
    RCC->AHB1ENR |= (1 << 1);    // GPIOB
    RCC->APB1ENR |= (1 << 21);    // I2C1

    // set PB8 and PB9 to alternate function mode (10)
    GPIOB->MODER &= ~(0b1111 << 16); // clear
    GPIOB->MODER |= (0b1010 << 16);   // set

    // set PB8 and PB9 to open-drain
    GPIOB->OTYPER |= (0b11 << 8);

    // pull-up pull-down mode
    GPIOB->PUPDR &= ~(0b1111 << 16); // clear
    GPIOB->PUPDR |= (0b0000 << 16);   // set (to zero)

    // set alternate function register
    GPIOB->AFR[1] &= ~(0xFF); // clear
    GPIOB->AFR[1] |= (0x44);   // set to AF4

    // sets peripheral clock frequency
    I2C1->CR2 &= ~(0b111111); // clear
    I2C1->CR2 |= (0b101010);   // 42 MHz
```

```

// set the SCL clock
I2C1->CCR &= ~(0b111111111111); // clear
I2C1->CCR |= (0b000011010010); // set to 100kHz

// set the rise time
I2C1->TRISE &= ~(0b111111); // clear
I2C1->TRISE |= (0b101011); // set to 43ns

// enable peripheral
I2C1->CR1 |= 0b1;
}

```

Results:

No actual results yet. Compiles correctly.

19:57:06 Build Finished. 0 errors, 0 warnings. (took 1s.491ms)

Learned:

- I2C Protocol has a bus, two wires, SCL (clock) and SDA (data). The master (STM32) controls the clock, and the slave (in this project, the MPU-6500 sensor) responds when addressed
- All devices on the bus hear transmissions, but only addressed device responds
- Uses **open-drain** with pull-up resistors so that multiple devices can be connected
- **Pull-up** resistors are active low, meaning the activation creates the short-circuit to ground and turn on
- **Pull-down** resistors are active high, meaning the activation creates the short-circuit to high voltage and turn on
- STM32 peripherals are clock gated to save power, this is why RCC was used to enable clocks for GPIOB and I2C1
- PB8 and PB9 were configured such that
 - MODER set to 10 (alternate function mode) for I2C peripheral usage
 - OTYPER set to open-drain because of I2C
 - PUPDR set to none because sensor already has these resistors
 - AFR (alternate function) set to AF4 because these are specifically mentioned in the STM32F4xxE datasheet
- I2C peripherals were configured such that
 - CR2 was the peripheral clock and it was told the APB1 clock speed of 42MHz
 - CCR is the SCL clock and it was set to 100kHz
 - TRISE is the maximum rise time for the peripheral, set to 43ns
 - CR1 PE was set to 1 to enable

Task B: WHO_AM_I

Do: Create an I2C read function that reads the MPU-6500's device address and sends it back via UART Shell created in the last project.

Code:

i2c.c:

```
// function to read any register from an I2C device
uint8_t i2c_read(uint8_t device_address, uint8_t regular_address, uint8_t*
result)
{
    // create START condition
    I2C1->CR1 |= (1 << 8);
    uint32_t timeout = 10000;
    // checks SB if start condition created
    while(!(I2C1->SR1 & (1 << 0))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // set device register to write mode
    I2C1->DR = device_address << 1 | 0;
    timeout = 10000;
    // checks ADDR if address successfully sent and ACKed
    while(!(I2C1->SR1 & (1 << 1))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }
    // clear ADDR
    (void)I2C1->SR1;
    (void)I2C1->SR2;

    // set register address to read from
    I2C1->DR = regular_address;
    timeout = 10000;
    // check BTF (byte transfer finished)
    while(!(I2C1->SR1 & (1 << 2))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // create another START condition
    I2C1->CR1 |= (1 << 8);
    timeout = 10000;
    while(!(I2C1->SR1 & (1 << 0))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }
}
```

```

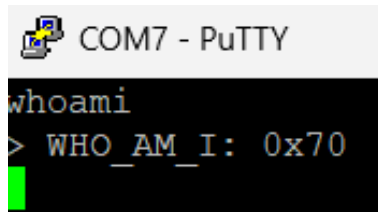
    // set device register to read mode
    I2C1->DR = device_address << 1 | 1;
    timeout = 10000;
    while(!(I2C1->SR1 & (1 << 1))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }
    // disable ACK because only reading one byte
    I2C1->CR1 &= ~(1 << 10);
    (void)I2C1->SR1;
    (void)I2C1->SR2;

    // generate STOP condition
    I2C1->CR1 |= (1 << 9);
    timeout = 10000;
    // check RXNE for data ready to be read
    while(!(I2C1->SR1 & (1 << 6))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }
    // read data register
    *result = I2C1->DR;
    return I2C_OK;
}

main.c:
else if(strcmp((char*)cmd, "whoami") == 0)
{
    uint8_t result = 0;
    char msg[32];
    uint8_t err = i2c_read(0x68, 0x75, &result);
    if(err != I2C_OK)
    {
        snprintf(msg, sizeof(msg), "> Error\r\n");
        HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
    }
    else
    {
        snprintf(msg, sizeof(msg), "> WHO_AM_I: 0x%02X\r\n", result);
        HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
    }
}

```

Results:



```
COM7 - PuTTY
whoami
> WHO_AM_I: 0x70
```

Learned:

- How to write a complete I2C read function
- Read the WHO_AM_I register (standard in MPU sensors) and got 0x70 back. This confirms the GPIO, peripheral config, and bus protocol all works
- There needs to be a START before writing and reading
- The read/write bits are the LSB of the address byte, 0 for write and 1 for read
- A status flag is polled (wait for potential timeout) in SR1 after each step
- NACK before last byte tells slave (MPU) to not send more data

Task C: Sensor Init

Do: Wake the MPU from sleep, configure accel/gyro range and sample rate

Code:

i2c.c:

```
uint8_t i2c_write(uint8_t device_address, uint8_t regular_address, uint8_t
byte_to_write)
{
    // create START condition
    I2C1->CR1 |= (1 << 8);
    uint32_t timeout = 10000;
    while(!(I2C1->SR1 & (1 << 0))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // set device register to write mode
    I2C1->DR = device_address << 1 | 0;
    timeout = 10000;
    // checks ADDR if address successfully sent and ACKed
    while(!(I2C1->SR1 & (1 << 1))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }
    // clear ADDR
    (void)I2C1->SR1;
    (void)I2C1->SR2;

    // set register address to write
    I2C1->DR = regular_address;
    timeout = 10000;
    // check BTF (byte transfer finished)
    while(!(I2C1->SR1 & (1 << 2))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // set byte to write
    I2C1->DR = byte_to_write;
    timeout = 10000;
    // check BTF (byte transfer finished)
    while(!(I2C1->SR1 & (1 << 2))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // generate STOP condition
    I2C1->CR1 |= (1 << 9);
```

```

        return I2C_OK;
    }

mpu.c:
#include "mpu.h"

#define MPU_ADDR 0x68

void mpu_init(void)
{
    // wake up MPU from sleep
    i2c_write(MPU_ADDR, 0x6B, 0x00);
    // configure sensor registers
    i2c_write(MPU_ADDR, 0x1B, 0x00);    // gyro
    i2c_write(MPU_ADDR, 0x1C, 0x00);    // accel
}

mpu.h:
#ifndef MPU_H
#define MPU_H

#include "stm32f4xx.h"
#include "i2c.h"

void mpu_init(void);

#endif

```

Results:

No results, but compiles correctly

22:00:11 Build Finished. 0 errors, 0 warnings. (took 1s.616ms)

Learned:

- Writing does not need multiple STARTs
- Created specific "mpu.c" and "mpu.h" files for the MPU 6500 sensor
- How to wake up the sensor by clearing power management bit
- Setting gyroscope and accelerometer to default state requires clearing bits

Task D: Burst Read

Do: Read all 14 bytes of sensor data in one transaction and convert raw values to physical units (acceleration, temperature, gyro)

Code:

main.c:

```

    else if(strcmp((char*)cmd, "read") == 0)
    {
        int16_t data[7];
        char msg[64];
        uint8_t err = mpu_read(data);
        if(err != I2C_OK)
        {
            snprintf(msg, sizeof(msg), "> Error\r\n");
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
        }
        else
        {
            MPU_Data mpu_data;
            mpu_convert(data, &mpu_data);

            snprintf(msg, sizeof(msg), "> Accel X: %.2f\r\n",
mpu_data.accel_x);
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);

            snprintf(msg, sizeof(msg), "> Accel Y: %.2f\r\n",
mpu_data.accel_y);
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);

            snprintf(msg, sizeof(msg), "> Accel Z: %.2f\r\n",
mpu_data.accel_z);
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);

            snprintf(msg, sizeof(msg), "> Temp (C): %.2f\r\n",
mpu_data.temp_c);
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);

            snprintf(msg, sizeof(msg), "> Gyro X: %.2f\r\n",
mpu_data.gyro_x);
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);

            snprintf(msg, sizeof(msg), "> Gyro Y: %.2f\r\n",
mpu_data.gyro_y);
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);

            snprintf(msg, sizeof(msg), "> Gyro Z: %.2f\r\n",
mpu_data.gyro_z);
            HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
        }
    }

```

```

    }
}

i2c.c:
uint8_t i2c_read_burst(uint8_t device_address, uint8_t regular_address,
uint8_t* buffer, uint8_t length)
{
    // create START condition
    I2C1->CR1 |= (1 << 8);
    uint32_t timeout = 10000;
    // checks SB if start condition created
    while(!(I2C1->SR1 & (1 << 0))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // set device register to write mode
    I2C1->DR = device_address << 1 | 0;
    timeout = 10000;
    // checks ADDR if address successfully sent and ACKed
    while(!(I2C1->SR1 & (1 << 1))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }
    // clear ADDR
    (void)I2C1->SR1;
    (void)I2C1->SR2;

    // set register address to read from
    I2C1->DR = regular_address;
    timeout = 10000;
    // check BTF (byte transfer finished)
    while(!(I2C1->SR1 & (1 << 2))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // create another START condition
    I2C1->CR1 |= (1 << 8);
    timeout = 10000;
    while(!(I2C1->SR1 & (1 << 0))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // set device register to read mode
    I2C1->DR = device_address << 1 | 1;
    timeout = 10000;
    while(!(I2C1->SR1 & (1 << 1))) {
        if(--timeout == 0)

```

```

        return I2C_ERROR;
    }
    // clear ADDR
    (void)I2C1->SR1;
    (void)I2C1->SR2;

    // read each byte
    for(int i = 0; i < length-1; i++)
    {
        timeout = 10000;
        // check RXNE for data ready to be read
        while(!(I2C1->SR1 & (1 << 6))) {
            if(--timeout == 0)
                return I2C_ERROR;
        }
        // read data register
        buffer[i] = I2C1->DR;
    }

    // disable ACK for last byte
    I2C1->CR1 &= ~(1 << 10);
    (void)I2C1->SR1;
    (void)I2C1->SR2;

    // generate STOP condition
    I2C1->CR1 |= (1 << 9);
    timeout = 10000;
    // check RXNE for data ready to be read
    while(!(I2C1->SR1 & (1 << 6))) {
        if(--timeout == 0)
            return I2C_ERROR;
    }

    // read last byte
    buffer[length-1] = I2C1->DR;

    // enable ACK
    I2C1->CR1 |= (1 << 10);
    return I2C_OK;
}

mpu.c:
uint8_t mpu_read(int16_t* data)
{
    // reads all 14 bytes for gyro, temp, and accel data
    uint8_t buffer[14];
    if (i2c_read_burst(MPU_ADDR, 0x3B, buffer, 14) == 0)
    {
        for(int i = 0; i < 14; i+=2)

```

```

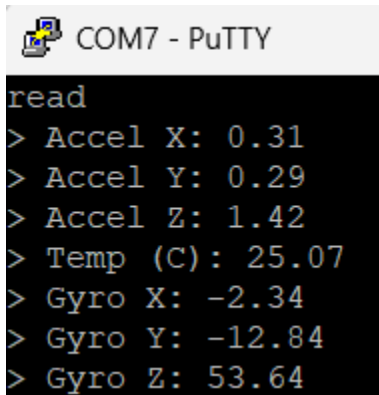
        {
            data[i/2] = buffer[i+1] | buffer[i] << 8;
        }

        return I2C_OK;
    }
    else
        return I2C_ERROR;
}

void mpu_convert(int16_t* raw, MPU_Data* out)
{
    // converts MPU data into physical units
    out->accel_x = raw[0] / 16384.0f;
    out->accel_y = raw[1] / 16384.0f;
    out->accel_z = raw[2] / 16384.0f;
    out->temp_c = (raw[3] / 333.87f) + 21.0f;
    out->gyro_x = raw[4] / 131.0f;
    out->gyro_y = raw[5] / 131.0f;
    out->gyro_z = raw[6] / 131.0f;
}

```

Results:



```

COM7 - PuTTY
read
> Accel X: 0.31
> Accel Y: 0.29
> Accel Z: 1.42
> Temp (C): 25.07
> Gyro X: -2.34
> Gyro Y: -12.84
> Gyro Z: 53.64

```

Learned:

- Reading many bytes in one transaction requires ACKing each received byte and then NACKing the last received byte
- Assembling the MPU data values required converting the 14 bytes into 7 signed integer (int16_t) values
- How to convert the raw MPU data into actual units by looking at the MPU 6500 datasheet
- Added ACK re-enable to end of i2c read functions so that data can be continuously read
- Burst read (14 bytes at once) is much more efficient than 14 individual reads (don't have to START each time)

Task E: Stream read

Do: Add a “stream” command to get frequent readings of MPU sensor data, and “stop” command to end the stream of data

Code:**main.c:**

Inside main(void) while loop:

```
// BLOCKING
// need timers to create non-blocking
if(streaming == 1)
{
    print_mpu_data();
    HAL_Delay(1000);
}
```

Inside process_command(cmd) function:

```
// reads accel, temp, and gyro data from MPU
else if(strcmp((char*)cmd, "read") == 0)
{
    print_mpu_data();
}
// constantly streams MPU sensor data
else if(strcmp((char*)cmd, "stream") == 0)
{
    streaming = 1;
    char msg[64];
    snprintf(msg, sizeof(msg), "> Streaming MPU Data\r\n");
    HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
}
// stops stream of MPU sensor data
else if(strcmp((char*)cmd, "stop") == 0)
{
    streaming = 0;
    char msg[64];
    snprintf(msg, sizeof(msg), "> Stopping Data Stream\r\n");
    HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
}
```

Results:

```

stream
> Streaming MPU Data
> Accel X: -0.08
> Accel Y: 0.03
> Accel Z: 1.04
> Temp (C): 24.93
> Gyro X: -3.50
> Gyro Y: 8.60
> Gyro Z: -0.95
> Accel X: -0.09
> Accel Y: 0.03
> Accel Z: 1.04
> Temp (C): 25.07
> Gyro X: -3.34
> Gyro Y: 8.82
> Gyro Z: -1.05
s> Accel X: -0.08
> Accel Y: 0.03
> Accel Z: 1.04
> Temp (C): 24.98
> Gyro X: -3.05
> Gyro Y: 8.89
> Gyro Z: -1.58
t> Accel X: -0.09
> Accel Y: 0.03
> Accel Z: 1.04
> Temp (C): 25.12
> Gyro X: -3.53
> Gyro Y: 9.21
> Gyro Z: -1.25
o> Accel X: -0.07
> Accel Y: 0.03
> Accel Z: 1.04
> Temp (C): 25.27
> Gyro X: -3.31
> Gyro Y: 8.68
> Gyro Z: -1.24
p> Accel X: -0.07
> Accel Y: 0.03
> Accel Z: 1.04
> Temp (C): 25.46
> Gyro X: -3.46
> Gyro Y: 9.00
> Gyro Z: -1.24
> Stopping Data Stream

```

Learned:

- Adding the stream was possible by implementing a new “streaming” flag. This flag would have MPU data be fetched when set to 1, and nothing would happen when set to 0.
- This added a new issue: The “stream” function floods the console AND is **blocking** (uses polling). This is shown when typing the “stop” command, each letter of that command is processed as each MPU data stream is printed. This takes multiple seconds for that command to process
- To fix this, need to implement **timers** to allow for **non-blocking** streaming of sensor data
- This will be implemented in the next project

What was learned overall:

- Project successfully added to previous UART Shell project by adding new commands “whoami” “read” “stream” and “stop” to read from an I2C connected MPU sensor
- Used bare-metal I2C communication by configuring registers appropriately based on STM32 F4xxE datasheet
- I2C has master (STM32) and slave (MPU-6500) devices, SCL clock line, and SDA data line. The master controls the clock. Address bits, register addresses, data bytes, ACKs, etc travel on the data line. Both are open-drain, meaning many devices can be connected at once
- There is a START before read and write, with read requiring two START conditions
- After each step for reading and writing, there is a status register flag that confirms what has been done, this is polled with a timeout meaning the previous step did not work
- The slave sends ACK after it receives an address, register, or data byte, and the master sends a NACK after the final byte to tell the slave to stop sending data
- The implementation of the I2C protocol allows for real-time readings of data from the MPU-6500
- Currently, streaming the data is blocking, meaning it takes away from processing new commands for large chunks of time. This needs to be changed with timers and interrupts with the next project.
- One of the biggest bugs I faced in this project was that I forgot to re-enable the ACK bit after single-byte reads. This bug made it so multiple commands could not be processed on startup. To fix this, I added ACK re-enable to the end of the `i2c_init` and `read` functions in the `i2c.c` file.
- In all, this project taught me the core concepts of the I2C communication protocol and how it is configured on the register level

Project 1 - UART Shell

Nathaniel Ross

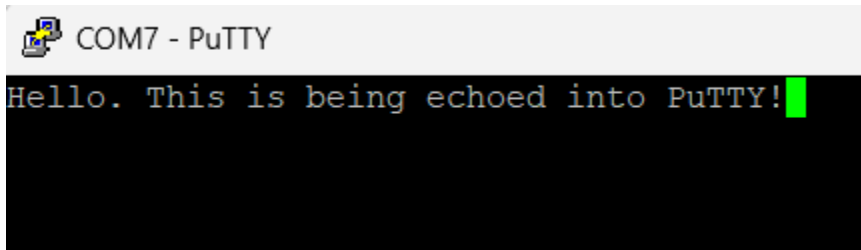
Task A: UART init + echo

Do: Write a polling loop that echoes back whatever typed in PuTTY console.

Code:

```
MX_USART2_UART_Init();
const char *message = "Hello";
uint8_t receive[1];
HAL_UART_Transmit(&huart2, (uint8_t*)message, 5, 10);
while (1)
{
    HAL_UART_Receive(&huart2, receive, 1, HAL_MAX_DELAY);
    HAL_UART_Transmit(&huart2, receive, 1, 10);
}
```

Results:



Learned:

- Current implementation uses **polling**: CPU waits for a certain amount of time and then processes an input/output. It asks “has a byte arrived?” in a loop.
- This uses a busy wait: HAL_MAX_DELAY makes the CPU wait until the next input, this wastes any additional processing capabilities for the CPU
- To solve this, we should use **interrupts**: stops whatever the CPU is doing, run a specific function, and come back. Function is called ISR (Interrupt Service Routine)

Task B: Interrupt-driven RX

Do: Use interrupts instead of polling for data transfer.

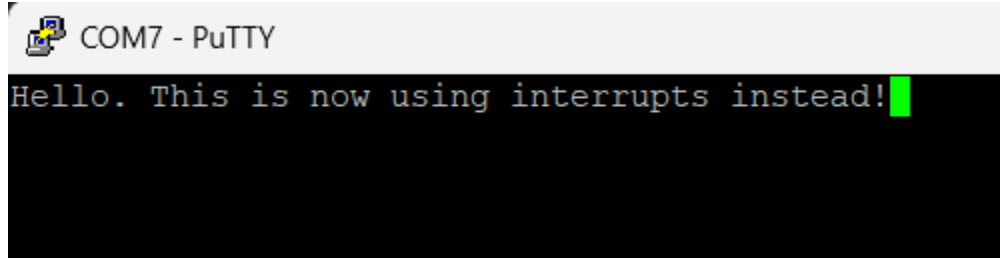
Code:

```
/* USER CODE BEGIN PV */
uint8_t receive[1];

/* USER CODE BEGIN 2 */
const char *message = "Hello";
HAL_UART_Transmit(&huart2, (uint8_t*)message, 5, 10);
HAL_UART_Receive_IT(&huart2, receive, 1);

/* USER CODE BEGIN 4 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance == USART2)
    {
        HAL_UART_Transmit_IT(huart, receive, 1);
        HAL_UART_Receive_IT(huart, receive, 1);
    }
}
```

Results:



Learned:

- Used interrupts to transmit messages. Now, the while(1) loop in main is empty, meaning the CPU has processing power to do other tasks
- Difference between Receive and Receive_IT: The first uses blocking (polling) mode and second uses non-blocking (interrupts) mode
- Receive_IT arms one byte for an interrupt, RxCpltCallback is a callback that uses the byte and echoes it, then re-arms the byte for another interrupt with Receive_IT
- In callback function, UART instance (USART2) is guarded from any other UART interrupt

Task C: Ring Buffer

Do: Create a ring buffer to store all bytes arriving from interrupt.

Code:

```
/* USER CODE BEGIN PTD */
#define BUFFER_SIZE 64

/* USER CODE BEGIN PV */
uint8_t receive[1];
uint8_t buffer[BUFFER_SIZE];
volatile uint8_t head;
volatile uint8_t tail;

/* Private function prototypes
-----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART2_UART_Init(void);
void ring_write(uint8_t byte);
uint8_t ring_read(void);
uint8_t ring_empty(void);

/* USER CODE BEGIN 2 */
head = 0;
tail = 0;
HAL_UART_Receive_IT(&huart2, receive, 1);

/* USER CODE BEGIN WHILE */
while (1)
{
    if(!ring_empty())
    {
        uint8_t byte = ring_read();
        HAL_UART_Transmit_IT(&huart2, &byte, 1);
    }
}

/* USER CODE BEGIN 4 */
void ring_write(uint8_t byte)
{
    // checks if buffer is not full
    if (head != (tail - 1 + BUFFER_SIZE) % BUFFER_SIZE)
    {
        // writes to head of ring buffer
        buffer[head] = byte;
        head = (head + 1) % BUFFER_SIZE; // wrap-around
    }
}
```

```

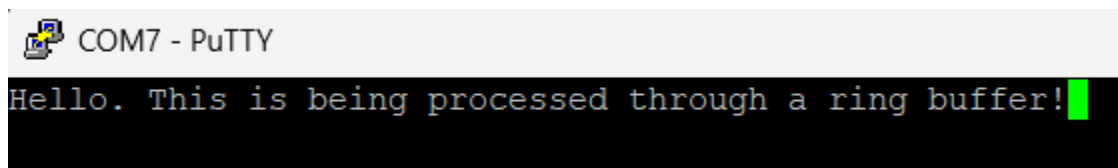
uint8_t ring_read(void)
{
    if (head == tail) // checks if empty buffer
    {
        return 0;
    }
    // reads current byte (tail) and advances
    else
    {
        uint8_t byte = buffer[tail];
        tail = (tail + 1) % BUFFER_SIZE; // wrap-around
        return byte;
    }
}

uint8_t ring_empty(void)
{
    // returns true if ring is empty
    if (head == tail)
        return 1;
    else
        return 0;
}

// callback function
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    // checks for USART 2
    if(huart->Instance == USART2)
    {
        ring_write(receive[0]);
        HAL_UART_Receive_IT(huart, receive, 1);
    }
}

```

Results:



Learned:

- Ring buffer used to manage fixed size, continuous stream of data. ISR routine doesn't block. This could be typing in the command prompt or a stream of LED commands
- Main loop uses buffer at its own pace, receiving and processing aren't mixed
- `buffer` sacrifices a slot to determine if it is full (so head doesn't wrap into tail)
- Full and empty ring conditions are handled, wrap-around feature handled for O(1) ops
- `volatile` keyword used to declare head and tail variables because they are used for interrupts, so they cannot be cached / optimized by the compiler

Task D: Line parser

Do: Create a line parser that accumulates bytes until a complete command arrives.

Code:

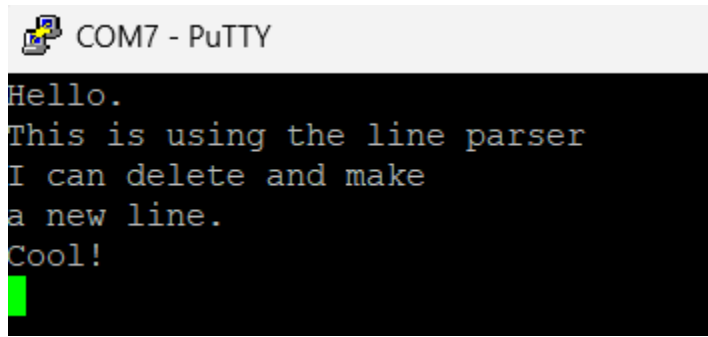
```
#define CMD_SIZE 32

/* USER CODE BEGIN 1 */
uint8_t cmd_buff[CMD_SIZE];
uint8_t cmd_index = 0;

while (1)
{
    if(!ring_empty())
    {
        uint8_t byte = ring_read();

        // New line
        if(byte == '\r' || byte == '\n')
        {
            cmd_buff[cmd_index] = '\0';
            uint8_t newline[] = "\r\n";
            HAL_UART_Transmit(&huart2, newline, 2, HAL_MAX_DELAY);
            cmd_index = 0;
            memset(cmd_buff, 0, CMD_SIZE);
        }
        // Backspace
        else if(byte == '\b' && cmd_index != 0)
        {
            cmd_index--;
            cmd_buff[cmd_index] = 0;
            uint8_t bs_seq[] = "\b \b";
            HAL_UART_Transmit(&huart2, bs_seq, 3, HAL_MAX_DELAY);
        }
        // Normal character, buffer isn't full
        else if(cmd_index < CMD_SIZE - 1)
        {
            cmd_buff[cmd_index] = byte;
            cmd_index++;
            HAL_UART_Transmit(&huart2, &byte, 1, HAL_MAX_DELAY);
        }
    }
}
```

Results:



```
COM7 - PuTTY
Hello.
This is using the line parser
I can delete and make
a new line.
Cool!
█
```

Learned:

- Line accumulation into buffer
- Can detect inputs, will be used in the next step to trigger commands
- Buffer overflow protection
- Clean buffer reset after command is sent
- This changed TX (transmission) to blocking (polling). This is fine because transmission only happens in response to user input (typing character and then echo). We would care about interrupt-driven TX if streaming large amounts of data continuously or had timing-critical work to do for transmission. For typing, transmitting one byte takes 86us at 115200 baud, so timing is a nonfactor.
- RX (receiving) is still interrupt driven

Task E: Command dispatcher

Do: Create a command dispatcher that, after newline case, compares the input to commands

Code:

```

/* USER CODE BEGIN 4 */
void process_command(uint8_t* cmd)
{
    uint8_t newline[] = "\r\n";
    HAL_UART_Transmit(&huart2, newline, 2, HAL_MAX_DELAY);

    if(strcmp((char*)cmd, "led on") == 0)
    {
        HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET);
        led_state = 1;
        uint8_t msg[] = "LED ON\r\n";
        HAL_UART_Transmit(&huart2, msg, sizeof(msg)-1, HAL_MAX_DELAY);
    }
    else if(strcmp((char*)cmd, "led off") == 0)
    {
        HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
        led_state = 0;
        uint8_t msg[] = "LED OFF\r\n";
        HAL_UART_Transmit(&huart2, msg, sizeof(msg)-1, HAL_MAX_DELAY);
    }
    else if(strcmp((char*)cmd, "status") == 0)
    {
        if(led_state)
        {
            uint8_t msg[] = "LED is ON\r\n";
            HAL_UART_Transmit(&huart2, msg, sizeof(msg)-1,
HAL_MAX_DELAY);
        }
        else
        {
            uint8_t msg[] = "LED is OFF\r\n";
            HAL_UART_Transmit(&huart2, msg, sizeof(msg)-1,
HAL_MAX_DELAY);
        }
    }
    else
    {
        uint8_t msg[] = "unknown cmd\r\n";
        HAL_UART_Transmit(&huart2, msg, sizeof(msg)-1, HAL_MAX_DELAY);
    }
}

```

Results:

```
status
LED is ON
led off
LED OFF
status
LED is OFF
█
```

Learned:

- How to create a strcmp-based state machine that executes real hardware actions such as turning on or off an LED
- One bug: If backspace is used, the command will not work. This can be fixed by sanitizing the input of any backspaces
- This can have much more possibilities, such as:
 - “help” command to list available commands
 - “led toggle” to toggle the led from its current state
 - “uptime” to see how long the LED has been on with HAL_GetTick()
 - Be able to control output pins on the STM32 to turn on external devices connected via breadboard. This could include changing the brightness of an LED with PWM, turning on a fan, typing and displaying to a LCD

What was learned overall:

- Difference between interrupt and polling
- Interrupt-driven UART RX (bytes arrive via ISR, don't block CPU)
- Ring buffer (decouples ISR from main loop, handles data burst safely)
- Line parser (accumulate characters, detect command completion)
- Command dispatcher (compare command via state machine to manipulate hardware)